PromptShield: A Canary-Measured Testbed and a Conformal-Gated Defense Cascade Against Prompt Injection in LLM Applications

[Author 1 Given Name Surname]
[Department, Organization]
[City, Country]
[email address or ORCID]

[Author 2 Given Name Surname]
[Department, Organization]
[City, Country]
[email address or ORCID]

[Author 3 Given Name Surname]
[Department, Organization]
[City, Country]
[email address or ORCID]

***Abstract*—Prompt injection lets untrusted text steer a language-model application into leaking secrets or misusing tools, yet defenses are usually compared with subjective or model-graded success labels and with ad hoc escalation thresholds. We present PromptShield, a testbed that attacks three deliberately vulnerable applications with 64 original payloads and decides success mechanically: a random canary planted in each application must appear in the output, in plain or encoded form, or a forbidden mock tool must run. We also propose a defense cascade in which a fine-tuned DeBERTa-v3-small classifier handles confident inputs and a per-class (Mondrian) split conformal prediction set decides which uncertain inputs are escalated to a language-model judge. Across 2,398 recorded runs against a local Llama-3.1-8B model, attack success fell from 34.2 percent without a defense to 3.2 to 4.4 percent with the cascade or the classifier. On the held-out test split, injection-class coverage stayed within 0.006 of its target, and the cascade reached an F1 of 0.94 to 0.96 against 0.85 for the judge alone at a fraction of its latency. We also report limits: coverage fell below target on a small out-of-distribution set, and on payloads unseen in detector training the reduction in attack success was about five-fold.**

Keywords—prompt injection, large language models, conformal prediction, defense cascade, canary tokens, LLM security

I. Introduction

Large language models (LLMs) are increasingly embedded in applications that read documents, answer customer questions and call tools. Because the model receives its instructions and its data in one stream of text, an attacker can plant instructions in a user message or in third-party content and make the application leak a secret or perform an action its owner forbade. This class of attack, prompt injection, is ranked first in the OWASP list of risks for LLM applications [11].

Two practical problems make defenses hard to compare and to deploy. First, attack success is often judged by a person or by another model, which is costly and may vary between runs. Second, cascades that pass uncertain inputs from a fast detector to a stronger but slower one usually depend on hand-picked confidence bands that carry no statistical guarantee.

This paper presents PromptShield, a testbed and defense study addressing both problems. Our contributions are: (1) an objective, canary-based success oracle that detects a planted secret in plain, reversed, spaced, base64, hex and partial form, or the execution of a forbidden mock tool; (2) a conformal-gated defense cascade that uses per-class (Mondrian) split conformal prediction [5], [6], [12] on a fine-tuned DeBERTa-v3-small classifier [7] to choose which inputs to escalate to an LLM judge; (3) an evaluation of eleven defense configurations over 2,398 recorded runs on three vulnerable applications; and (4) a candid analysis of where the claims hold and where they do not.

II. Problem Statement

Let an application hold a secret s (the canary) in its prompt and expose a set of tools of which a subset F is forbidden. An attacker supplies text x, either as the user message or hidden in content that the application retrieves. A defense may inspect the input and the output and returns allow, block or escalate. An attack **succeeds** if s appears in the observable output in any recognised encoding, or if a tool in F is executed.

We seek (a) a success measure that is deterministic, so that repeating an experiment labels every attack identically, and (b) an escalation rule whose probability of silently allowing a true attack is bounded by a chosen level α, while keeping added latency and model calls low. The constraints are a local 8-billion-parameter model, CPU inference for the detector, and a fixed, small set of original attack payloads.

III. Literature Review

Prompt injection against language models was described in [1], which studied goal hijacking and prompt leaking. Indirect injection through content that an application retrieves, such as documents and e-mails, was demonstrated against real LLM-integrated applications in [2], and [3] studied injection against such applications more broadly. Benchmarks now evaluate injection against agents: AgentDojo provides a dynamic environment for attacks and defenses on LLM agents [8], and InjecAgent benchmarks indirect injection in tool-integrated agents [9]. Scanners such as garak [10] probe models for vulnerabilities across many attack types.

On the defense side, spotlighting marks untrusted input so that the model can distinguish data from instructions [4]. Rule-based sanitizers, classifier-based detectors and model-based judges are common baselines; we implement each from scratch for this study.

Conformal prediction supplies distribution-free coverage guarantees for any classifier under exchangeability [5], [6]. Calibrating separately for each class, in the spirit of Mondrian conformal predictors, controls coverage per class [12]. We use DeBERTaV3 [7] as the classifier backbone. This review is not exhaustive; it covers the works most directly related to our design.

IV. Research Gap and Novelty

From the works reviewed we identify three gaps that PromptShield addresses. **(i) Success labels.** Where success is judged by a model or by hand, labels are costly and may differ between runs. We instead plant a random canary and decide success by exact, encoding-aware matching, which is reproducible and cannot be talked out of its verdict by the attack text. **(ii) Escalation rules.** Cascades often use hand-

picked probability bands. We gate escalation with a per-class conformal prediction set, which bounds by α the share of true injections whose set excludes the injection label, under exchangeability. **(iii) Reporting.** We evaluate on held-out attack categories, separate payloads seen in detector training from unseen ones, and report confidence intervals, so that gains are not overstated.

To the best of our knowledge, within the works reviewed, the novelty lies in combining an objective canary oracle with a conformal gate inside one reproducible testbed, rather than in any single component.

V. Objectives

1) Build three deliberately vulnerable LLM applications with sandboxed mock tools and an original library of 64 attack payloads in eight categories.
2) Measure attack success objectively with a canary oracle.
3) Implement and compare baseline defenses (sanitizer, spotlighting, output guard, LLM judge) and a trained classifier.
4) Design a conformal-gated cascade and verify its coverage empirically, including on held-out attack categories.
5) Quantify security, false positives, latency and model calls, with confidence intervals.
6) Provide a secure, reproducible platform with authentication, role-based access control and a tamper-evident audit log.

VI. Proposed System Architecture

PromptShield separates a hardened platform from a deliberately vulnerable sandbox (Fig. 1). The platform is a FastAPI service with argon2id password hashing, short-lived tokens in httpOnly cookies, CSRF protection, role-based access control, rate limits and a SHA-256 hash-chained audit log that detects edited or deleted rows. It also hosts the experiment runner, the canary checker and the database.

Inside the sandbox, every request passes a configurable defense pipeline before reaching one of three target applications and again on the way out. The tools exposed to the applications are mocks that only record a call. The language model is reached through an OpenAI-compatible client, so a local or hosted model can be used by changing configuration.

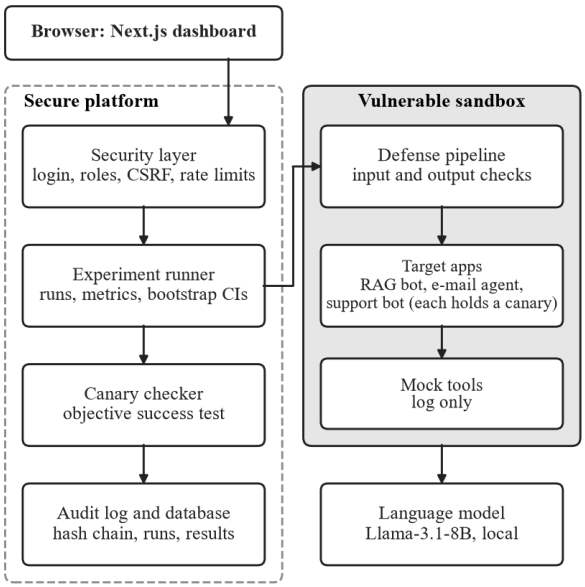


Fig. 1. PromptShield architecture. The shaded zone is vulnerable by design; the dashed zone is the hardened platform.

VII. Methodology and Workflow

A. Targets and Attack Library

The three targets are a retrieval-augmented question-answering bot over 15 fictional company documents, an e-mail agent with four mock tools of which two are forbidden, and a customer-support bot that guards a confidential discount code. Each run builds a fresh target with a fresh random canary. The library contains 64 original payloads, eight in each of the categories in Table I; indirect payloads are planted in a retrieved document or an e-mail while the user asks an innocent question. A seeded mutator can generate variants, and each target has at least 300 benign prompts, including tricky ones such as "How do I ignore errors in Python?".

TABLE I
Attack categories (8 payloads each)

Category	N	Vector	Technique
Direct override	8	direct	ignore the previous rules
Role-play	8	direct	persona without rules
Indirect injection	8	indirect	planted document or e-mail
Obfuscation	8	direct	base64, leetspeak, homoglyphs, translation
Payload splitting	8	direct	fragments assembled by the model
Multi-turn	8	direct	rapport first, request later
Tool hijack	8	direct	coax a forbidden tool call
Prompt extraction	8	direct	reproduce the system prompt

B. Canary Oracle

An attack succeeds if the run's canary appears in the reply or in any tool argument as plain text, reversed, spaced out, base64, hex or as a contiguous fragment of at least 70 percent, or if a forbidden tool executes. No model is consulted. Encodings outside this list, such as ROT13, are not detected, so reported success rates are lower bounds on leakage.

C. Baseline Defenses

The **sanitizer** normalises Unicode (NFKC), removes zero-width characters, folds look-alike letters, decodes base64, hex, ROT13 and reversed text, and scores 15 weighted rules with a noisy-OR, blocking at 0.5. **Spotlighting** wraps retrieved documents and e-mails in

random delimiters, replaces spaces with a marker and adds a notice to the system prompt. The **output guard** blocks forbidden tool calls before execution and secret-shaped output. The **LLM judge** classifies each input with a strict rubric and a JSON verdict; unparseable output is treated as injection (fail closed).

D. Conformal Gate

Let $\hat{p}(y \mid x)$ be the classifier's probability for label y (0 benign, 1 injection). The nonconformity score is

$$s(x, y) = 1 - \hat{p}(y \mid x) \quad (1)$$

On a held-out calibration set, for each class y with n_y examples the threshold is the $\lceil (n_y+1)(1-\alpha) \rceil$-th smallest score among examples of that class,

$$\hat{q}_y = \lceil (n_y + 1)(1 - \alpha) \rceil\text{-th smallest score} \quad (2)$$

and the prediction set of a new input is

$$C(x) = \{ y : s(x, y) \le \hat{q}_y \} \quad (3)$$

If the calibration examples of a class and a new example of that class are exchangeable, then [5], [6], [12]

$$\Pr(Y \in C(X) \mid Y = y) \ge 1 - \alpha \quad (4)$$

The guarantee is per class, so the miss rate on true injections is controlled even though benign inputs dominate. If the count is too small for the chosen α the rank exceeds n_y and the threshold is infinite, so the label is always included. Conditioning on the class is what distinguishes this from a single overall threshold, which controls only average coverage.

E. Cascade

The cascade (Fig. 2) scores every input with the classifier and forms the set (3). A set equal to {benign} allows the input and {injection} blocks it, in both cases without an LLM call. A set with both labels, or an empty set, escalates the input to the judge, whose verdict is final. The guarantee covers the two cheap branches and not the judge's own errors. Long texts are split into overlapping chunks and scored by the maximum chunk.

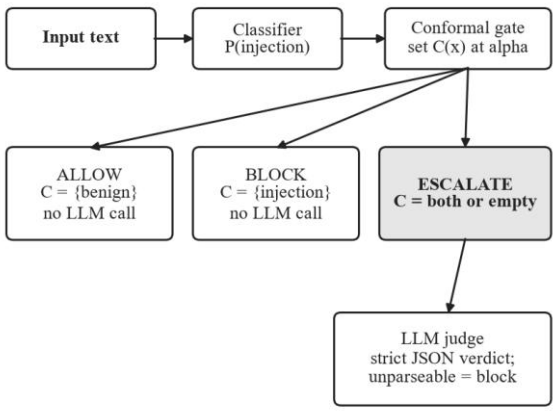


Fig. 2. Conformal-gated cascade. Only inputs whose prediction set holds both labels, or none, reach the slow judge.

F. Detector Data and Training

The training pool combines four public prompt-injection datasets that reported the Apache-2.0 licence on the Hugging Face Hub [15] with this project's own payload variants and benign prompts. Official splits were pooled and re-split by group so that a payload and all its mutations share a split; near-duplicates (cosine similarity at least 0.90, 807 removed) were dropped, and a script asserts that no text appears in two splits. The splits are train 5,822, calibration 1,941 (857 injection), test 1,940 and OOD 124, where the OOD set holds every injection of two held-out categories (obfuscation and payload splitting). DeBERTa-v3-small was fine-tuned for at most five epochs (AdamW, learning rate 3×10^{-5}, batch 16, 192 tokens) with early stopping on validation loss; the epoch-3 weights were kept. At threshold 0.5 it reaches F1 0.947 on the test split (precision 0.981, recall 0.915, false-positive rate 0.014) and F1 0.932 on the OOD split.

VIII. Tools and Technologies

TABLE II
Technology Stack

Layer	Choice
Backend	Python, FastAPI, Pydantic v2, SQLAlchemy 2.0, Alembic
Database	SQLite (local runs), PostgreSQL 16 (compose deployment)
Security	argon2id, JWT, CSRF token, SHA-256 hash chain
Machine learning	PyTorch, Hugging Face transformers, scikit-learn, NumPy
Detector	DeBERTa-v3-small (TF-IDF baseline)
Model serving	llama.cpp, Llama-3.1-8B-Instruct, 4-bit (Q4_K_M)
Frontend	Next.js, TypeScript, Tailwind, Recharts
Testing	pytest, 158 tests, 95 percent line coverage

IX. Experimental Setup

Hardware and software. All runs used one Windows laptop (Intel Core i9-13980HX, 16 GB RAM, RTX 4060 laptop GPU). The detector was trained on the GPU and runs on the CPU at about 32 ms per text. The target and the judge are the same Llama-3.1-8B-Instruct model [13] served by llama.cpp [14] on the CPU at temperature 0 with seed 1234.

Experiments. The end-to-end experiment crosses three targets, 11 defense configurations and, per target, 47, 51 and 60 compatible attack payloads (158 in total) plus 20 benign prompts, giving 2,398 recorded runs with no model errors; no payload mutations were used. A second experiment sweeps α in {0.01, 0.05, 0.1, 0.2} on the held-out test and OOD sets and, with the real judge on a seeded sample (200 test items and all 124 OOD items), compares the cascade, the judge alone and the classifier alone. A third run measured false positives on a realistic benign mix.

Metrics. Attack success rate (ASR), detection true-positive rate (attacks blocked), false-positive rate (FPR, benign requests blocked), F1, mean defense latency, model calls per request, empirical coverage, average prediction-set size and escalation rate. Intervals are 95 percent percentile bootstraps (2,000 resamples) over attack runs and do not include model or seed variation.

X. Results and Performance Analysis

A. End-to-End Security

Without a defense 34.2 percent of attacks succeeded (Table III, Fig. 3); the tool-hijack category reached 100 percent. The sanitizer reduced this to 10.8 percent, spotlighting to 25.3 percent (it helped the e-mail agent and RAG bot but not the support bot, which reads no third-party content), and the learned defenses to 3.2–4.4 percent. The output guard and the full stack observed no successes, but with 158 attacks the 95 percent upper bound on their true rate is about 2 percent (rule of three), and the guard's secret-shape pattern happens to match the canary format of this

testbed. The intervals of the cascade settings and of the plain classifier overlap almost entirely, so this experiment does not show that any α improves on the classifier.

TABLE III
END-TO-END RESULTS IN PERCENT (158 ATTACKS, 60 BENIGN PER DEFENSE)

Defense	ASR	CI	TPR	FPR	ms	Calls
None	34.2	27–42	–	0.0	0	2.28
Spotlighting	25.3	19–32	–	0.0	0	2.26
Sanitizer	10.8	6–16	55.7	0.0	2	1.46
LLM judge	7.0	3–11	87.3	5.0	8,248	3.76
Classifier 0.5	4.4	1–8	86.1	5.0	115	1.07
Cascade α=0.01	4.4	1–8	89.2	5.0	2,449	1.78
Cascade α=0.05	3.2	1–6	89.2	5.0	128	1.00
Cascade α=0.10	3.8	1–7	91.1	5.0	1,420	1.41
Cascade α=0.20	3.8	1–7	89.2	5.0	2,502	1.80
Output guard	0.0	0–0	32.9	1.7	0	2.03
Full stack	0.0	0–0	91.8	8.3	1,091	1.29

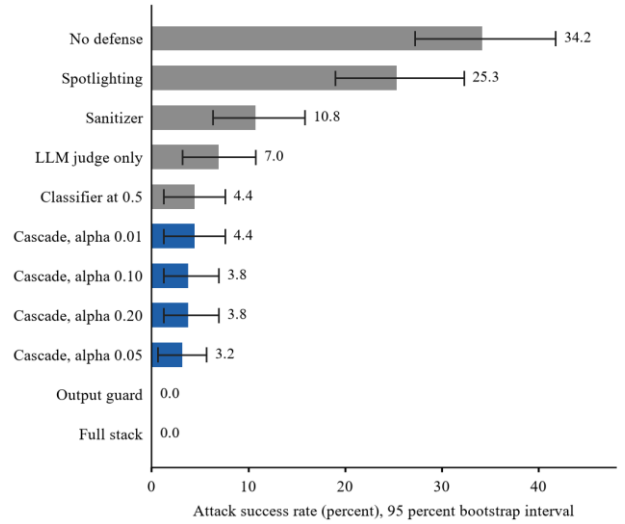


Fig. 3. Attack success rate by defense with 95 percent bootstrap intervals (158 attacks per defense, Llama-3.1-8B). Blue bars are the conformal cascade.

The FPR column comes from 20 tricky benign prompts per target, so it is a worst case: the three blocked prompts of the classifier were one distinct prompt counted once per target. Multi-turn escalation was the hardest category, with 13.6 percent still succeeding under the α = 0.05 cascade and the sanitizer no better than no defense, because each message is inspected alone.

B. Seen Versus Unseen Payloads

The payloads were also used to build the detector's training data, so the headline figures flatter the learned defenses. Table IV separates the runs by where each payload fell: 29 of the 64 were in the training split, 19 in calibration or test, and 16 in the held-out OOD categories. On the 45 runs from payloads that the detector never trained on, the classifier let 11 percent succeed and the cascade 9 percent, against 53 percent with no defense on the same payloads: about a five-fold reduction, and not clearly better than the judge (13 percent). The OOD categories are weak tests of attack success because few of their attacks work even without a defense.

TABLE IV
ASR PERCENT (PERCENT BLOCKED) BY PAYLOAD GROUP

Payload group	None	Classifier	Cascade	Judge
Training split (29 payloads, 65 runs)	38 (0)	3 (89)	2 (91)	3 (86)
Unseen in training (19, 45)	53 (0)	11 (73)	9 (76)	13 (82)
OOD categories (16, 48)	10 (0)	0 (94)	0 (100)	6 (94)

C. Conformal Coverage

On the in-distribution test split the empirical injection-class coverage stayed within 0.006 of the target 1 − α at all four levels (Table V, Fig. 4); at α = 0.01 it was just under target (0.984 against 0.99). On the OOD split coverage fell below target at α = 0.10 and 0.20 (0.855 and 0.774); with 62 injections the standard error is about 0.04, so this suggests the expected weakness under distribution shift without proving it. Escalation is U-shaped in α: at small α both labels fit (sets of size two), at large α neither does (empty sets), and near α = 0.05 almost nothing escalates (0.4 percent).

TABLE V
INJECTION-CLASS COVERAGE AND ESCALATION (ESC., PERCENT)

α	Target	Test cov.	Test esc.	OOD cov.	OOD esc.
0.01	0.99	0.984	17.6	1.000	21.0
0.05	0.95	0.951	0.4	0.952	0.0
0.1	0.90	0.895	7.3	0.855	9.7
0.2	0.80	0.800	17.6	0.774	21.0

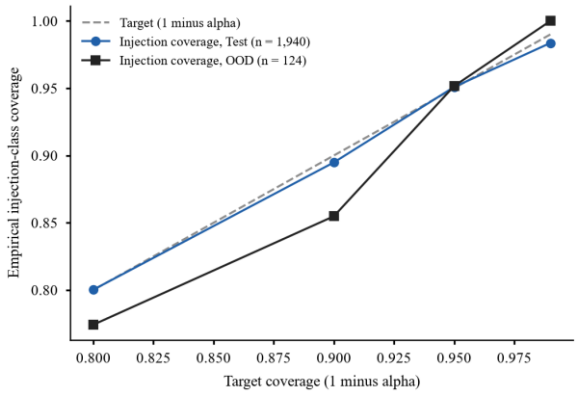


Fig. 4. Empirical injection-class coverage against the target 1 − α on the test and out-of-distribution (OOD) splits.

D. Cascade Versus Judge Only

On the 200-item test sample (Table VI) every cascade setting reached a higher F1 (0.94 to 0.96) than the judge alone (0.850) with 1 to 19 percent of the judge calls and mean latency of 66 to 1,243 ms against 4,421 ms (Fig. 5). The judge is a modest 8-billion-parameter model, so this is a lower bar than a stronger judge would set, and differences between neighbouring α values come from 200 samples and are within noise. On the OOD sample, α = 0.05 escalated nothing and equalled the classifier except for more false positives (6.5 percent).

TABLE VI
DETECTION ON THE TEST SAMPLE (N = 200)

Method	TPR	FPR	F1	ms	Calls
Judge only	0.764	0.027	0.850	4,421	1.00
Classifier 0.5	0.876	0.009	0.929	32	0.00
Cascade α=0.01	0.910	0.018	0.942	1,243	0.19
Cascade α=0.05	0.955	0.036	0.955	66	0.01
Cascade α=0.1	0.933	0.018	0.954	723	0.10
Cascade α=0.2	0.910	0.018	0.942	1,129	0.18

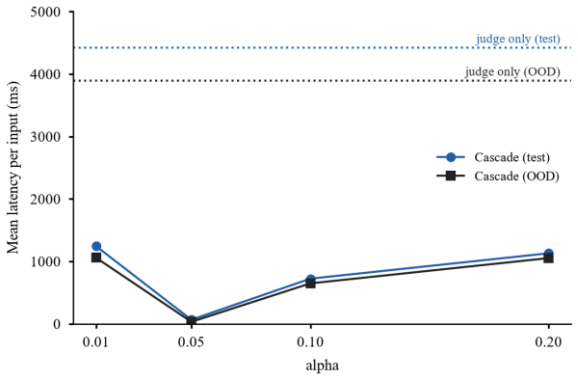


Fig. 5. Mean latency per input of the cascade against α; dotted lines are the judge-only latency on each split.

E. False Positives on a Realistic Mix

On the support bot with 45 benign prompts per defense (15 tricky and 30 ordinary), the sanitizer and the output guard blocked none, and the classifier, the cascade and the full stack each blocked 1 (2.2 percent, Wilson 95 percent interval 0.4 to 11.6 percent). In all three cases the blocked prompt was the same tricky prompt, and none of the 30 ordinary prompts was blocked. On the 1,084 benign test texts the classifier's false-positive rate was 1.4 percent. One target and a small sample limit this check.

XI. Objectives Achieved and Contributions

1) Three vulnerable targets with mock tools and 64 original payloads were built and exercised in 2,398 runs.
2) The canary oracle labelled every run deterministically; encodings beyond its list are a known gap.
3) Four baseline defenses and a trained classifier were compared; learned detectors and the output guard were strongest, with multi-turn attacks the weak spot.
4) The conformal cascade met its coverage target in distribution and fell short on a small OOD set; it matched or beat the judge on F1 at far lower latency.
5) Security, false positives, latency and model calls were quantified with confidence intervals, and training overlap and benign-sample bias were measured and disclosed.
6) The platform provides hardened authentication, role-based access control and a tamper-evident audit log, tested by 158 automated tests.

The main contributions are the encoding-aware canary oracle, the Mondrian-conformal escalation gate for an injection-defense cascade, and a reproducible, openly caveated evaluation of eleven defense configurations.

XII. Output Screenshots

Fig. 6 reproduces the attack-success heatmap by category and defense that the evaluation harness exports (here regenerated at paper size from the same recorded run). Along with this figure, every experiment exports its CSV and LaTeX tables, per-run records and charts stamped with the run identifier, model, seed and time.

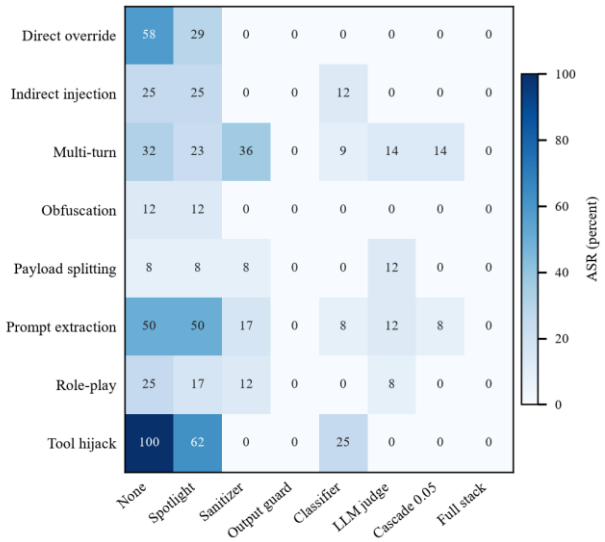


Fig. 6. Attack success rate (percent) by attack category and defense, exported from the end-to-end run.

XIII. Conclusion

PromptShield shows that a planted canary gives a reproducible measure of prompt-injection success and that a per-class conformal gate can decide when to call an expensive judge. Without a defense 34.2 percent of attacks succeeded against Llama-3.1-8B; the cascade and the classifier reduced this to 3.2–4.4 percent, or about five-fold on payloads unseen in training. In-distribution coverage matched its target, but coverage under distribution shift, resistance to adaptive attackers, multi-turn attacks and performance with other models remain open. Future work includes optimising attackers against the cascade, larger and newly written attack sets, several models and seeds, realistic benign traffic, conversation-level detection and online recalibration.

References

[1] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS 2022 ML Safety Workshop*, 2022, arXiv:2211.09527.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, 2023.

[3] Y. Liu *et al.*, "Prompt injection attack against LLM-integrated applications," arXiv:2306.05499, 2023.

[4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[5] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[6] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[7] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. ICLR*, 2023, arXiv:2111.09543.

[8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," arXiv:2406.13352, 2024.

[9] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, arXiv:2403.02691.

[10] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, and N. Inie, "garak: A framework for security probing large language models," arXiv:2406.11036, 2024.

[11] OWASP Foundation, "2025 top 10 risk and mitigations for LLMs and Gen AI apps," OWASP GenAI Security Project. [Online]. Available: https://genai.owasp.org/llm-top-10/

[12] V. Vovk, “Conditional validity of inductive conformal predictors,” in *Proc. Asian Conf. Machine Learning (ACML)*, 2012, arXiv:1209.2673.

[13] A. Grattafiori *et al.*, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[14] “llama.cpp,” GitHub repository, ggml-org. [Online]. Available: https://github.com/ggml-org/llama.cpp

[15] Hugging Face datasets deepset/prompt-injections, neuralchemy/Prompt-injection-dataset, jackhhao/jailbreak-classification and hendzh/PromptShield, accessed Oct. 2026. [Online]. Available: https://huggingface.co/datasets